

ЛР27 (3 пары / 6 часов)

«Диагностика и Support Report: проверка системы и сбор отчёта»

0) Итог ЛР27 (что должно быть готово)

✓ Добавлены команды:

- 28) Диагностика
- 29) Собрать отчёт для поддержки

✓ Диагностика показывает:

- версию приложения
- роль
- режим хранения
- папки и доступ к ним
- файлы config.json и tasks.json
- количество задач активных/архивных/в корзине
- наличие логов

✓ Support Report сохраняется в папку `reports`

✓ Есть отчёт `docs/LR27_Report.md`

✓ Минимум 3 коммита

1) Теория (важное)

1.1 Диагностика

Диагностика — это проверка, что программа может работать нормально.

1.2 Support Report

Support Report — файл, который можно отправить, если приложение работает неправильно.

2) Практика. Пара 1 (2 часа) — DiagnosticsService

Шаг 1. Создать папку Support

В `TaskTracker.App` создать папку:

- `Support`
-

Шаг 2. Создать DiagnosticsService.cs

Создать файл:

- `TaskTracker.App → Support → DiagnosticsService.cs`

Вставить:

```
using TaskTracker.App.Config;
using TaskTracker.Core.Services;

namespace TaskTracker.App.Support;

public class DiagnosticsService
{
    private readonly string _baseDir;
    private readonly string _configPath;
    private readonly AppConfig _cfg;

    private readonly string _dataFolder;
    private readonly string _logsFolder;
    private readonly string _backupsFolder;
    private readonly string _exportsFolder;
    private readonly string _reportsFolder;
    private readonly string _dataFilePath;

    public DiagnosticsService(
```

```

string baseDir,
string configPath,
    AppConfig cfg,
string dataFolder,
string logsFolder,
string backupsFolder,
string exportsFolder,
string reportsFolder,
string dataFilePath)
{
    _baseDir = baseDir;
    _configPath = configPath;
    _cfg = cfg;

    _dataFolder = dataFolder;
    _logsFolder = logsFolder;
    _backupsFolder = backupsFolder;
    _exportsFolder = exportsFolder;
    _reportsFolder = reportsFolder;
    _dataFilePath = dataFilePath;
}

public List<string>Run(TaskService service)
{
    var lines =new List<string>();

    lines.Add("=== Diagnostics ===");
    lines.Add($"DateTime: {DateTime.Now:yyyy-MM-dd HH:mm:ss}");
    lines.Add($"BaseDir: {_baseDir}");

    var version =typeof(DiagnosticsService).Assembly.GetName().Version?.ToString() ?? "unknown";
    lines.Add($"AppVersion: {version}");

    lines.Add($"Role: {_cfg.Role}");
}

```

```

        lines.Add($"StorageMode: {_cfg.StorageMode}");

        lines.Add($"ConfigPath exists: {File.Exists(_configPa
th)}}");
        lines.Add($"TasksFile exists: {File.Exists(_dataFileP
ath)}}");

        lines.Add("");
        lines.Add("Folders check:");
        lines.Add(CheckFolder("Data", _dataFolder));
        lines.Add(CheckFolder("Logs", _logsFolder));
        lines.Add(CheckFolder("Backups", _backupsFolder));
        lines.Add(CheckFolder("Exports", _exportsFolder));
        lines.Add(CheckFolder("Reports", _reportsFolder));

        lines.Add("");
        lines.Add("Tasks summary:");
        lines.Add($"Total:    {service.GetAll().Count}");
        lines.Add($"Active:  {service.GetAllActive().Coun
t}");
        lines.Add($"Trash:    {service.GetTrash().Count}");
        lines.Add($"Archive: {service.GetArchive().Count}");

var todayLog = Path.Combine(_logsFolder,$"app_{DateTime.Now:y
yyy-MM-dd}.log");
var todayErr = Path.Combine(_logsFolder,$"errors_{DateTime.No
w:yyyy-MM-dd}.log");

        lines.Add("");
        lines.Add("Logs:");
        lines.Add($"Today log exists: {File.Exists(todayLo
g)}}");
        lines.Add($"Today errors exists: {File.Exists(todayEr
r)}}");

        lines.Add("=====");

```

```

return lines;
    }

private static string CheckFolder(string name, string path)
{
    try
    {
        Directory.CreateDirectory(path);

        var testFile = Path.Combine(path, ".write_test.tmp");
        File.WriteAllText(testFile, "test");
        File.Delete(testFile);

        return $"{name}: OK ({path})";
    }
    catch (Exception ex)
    {
        return $"{name}: ERROR ({path}) ->{ex.Message}";
    }
}
}

```

Шаг 3. Коммит №1

Сообщение:

- feat: add DiagnosticsService

Commit → Push.

3) Практика. Пара 2 (2 часа) — команды 28 и 29

Шаг 4. Добавить пункты меню

В `ConsoleUi.PrintMenu()` добавить:

```
Console.WriteLine("28) Диагностика");  
Console.WriteLine("29) Собрать отчёт для поддержки");
```

Шаг 5. Подключить using

В `Program.cs` добавить:

```
using TaskTracker.App.Support;
```

Шаг 6. Создать объект diagnostics

После того как созданы пути папок и dataFilePath:

```
var diagnostics =new DiagnosticsService(  
    baseDir,  
    configPath,  
    cfg,  
    dataFolder,  
    logsFolder,  
    backupsFolder,  
    exportsFolder,  
    reportsFolder,  
    dataFilePath  
);
```

Шаг 7. Команда 28 (диагностика)

Добавить:

```
if (input == "28")  
{
```

```

        SafeRunner.Run("DIAGNOSTICS", logger, () =>
        {
            var lines = diagnostics.Run(service);

            foreach (var line in lines)
                Console.WriteLine(line);

            logger.Info("DIAGNOSTICS ran");
        });

        continue;
    }
}

```

Шаг 8. Команда 29 (Support Report)

Добавить:

```

if (input == "29")
{
    SafeRunner.Run("SUPPORT_REPORT", logger, () =>
    {
        Directory.CreateDirectory(reportsFolder);

        var lines = diagnostics.Run(service);

        var file = Path.Combine(
            reportsFolder,
            $"SupportReport_{DateTime.Now:yyyy-MM-dd_HH-mm-ss}.txt"
        );

        File.WriteAllLines(file, lines);

        Console.WriteLine("Отчёт создан: " + file);
        logger.Info($"SUPPORT_REPORT created file=\"{file}");
    });
}

```

```
\");  
});  
  
continue;  
}
```

Шаг 9. Коммит №2

Сообщение:

- `feat: add commands 28/29 diagnostics and report`

Commit → Push.

4) Практика. Пара 3 (2 часа) — проверка и отчёт

Шаг 10. Проверка 28

Запустить программу → команда 28 → проверить вывод.

Шаг 11. Проверка 29

Команда 29 → убедиться, что файл появился в папке `reports`.

Шаг 12. Коммит №3

Сообщение:

- `feat: finalize diagnostics and support report`

Commit → Push.

5) Отчёт ЛР27

`docs/LR27_Report.md`

```
# Отчёт ЛР27 – диагностика и Support Report
```


Что сделано

- Добавлен `DiagnosticsService`
- Добавлены команды:
 - 28) Диагностика
 - 29) Собрать отчёт для поддержки
- Support Report сохраняется в папку `reports`
- Результат логируется

Как проверить

- 1) Запустить приложение
- 2) Выполнить 28 -> увидеть диагностику
- 3) Выполнить 29 -> увидеть созданный файл в `reports`

Коммит:

- `docs: add LR27 report`